21. Complexity, Recursion, and Big-O Practice

21. Complexity, Recursion, and Big-O Practice

Phase: Phase 4 — Algorithms and data structures
Estimated time: 1–2 days
Slug: `21-complexity-recursion`

Learning objectives

- Measure time/space with Big-O
- Trace recursive call trees
- Convert recursion to iteration when needed
- Estimate cost before coding

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Big-O intuition

Big-O describes how runtime/memory grow as input size n grows. Common classes: O(1), O(log n), O(n), O(n log n), O(n^2), O(2^n).

```
// O(n)
func Sum(xs []int) int {
    s := 0
    for _, v := range xs { s += v }
    return s
}
```

Nested loops over the same n often mean O(n^2). Sorting good algorithms are O(n log n).

Recursion

```
func Fib(n int) int {
    if n < 2 { return n }
    return Fib(n-1) + Fib(n-2) // exponential without memo
}
```

Always define base cases. Watch stack depth — Go stacks grow, but recursion depth still matters.

Practice

For every algorithm you write: state expected complexity, then verify with benchmarks on growing n.

Common mistakes (especially from Python instincts)

- Calling naive Fib-like recursion in production
- Confusing average vs worst case

- Ignoring space complexity of maps/slices

Exercises

Work in `exercises/21-complexity-recursion/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/21-complexity-recursion/` until you have an honest failing attempt.

Self-check

1. What does O(n log n) typically describe?
2. Why is naive Fib exponential?
3. How do you empirically check complexity?

Further reading

- https://en.wikipedia.org/wiki/Big_O_notation